

Module 09

LangGraph + Agent Orchestration

Building AI agents that use tools, reason, and take actions

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is an AI Agent?

A chatbot just responds to messages. An **agent** can take **actions**. It decides what tool to use (search the web, query a database, send an email), uses it, observes the result, and decides what to do next. This loop continues until it has accomplished the goal.

Example: You say 'Show me unprocessed articles from last week'. The agent decides to call a database query tool, gets the results, formats them, and returns the answer. You then say 'Approve idea #3'. The agent calls the `approve_idea` tool.

2. LangChain Tools — The `@tool` Decorator

A **tool** is a Python function that the LLM can decide to call. LangChain's `@tool` decorator converts any function into a tool by reading its name, docstring, and type annotations.

```
from langchain_core.tools import tool
from typing import Optional

@tool
def get_weather(city: str, units: str = "celsius") -> str:
    """Get the current weather for a city.
    city: The city name (e.g. 'Mumbai', 'Delhi').
    units: 'celsius' or 'fahrenheit'"""
    # Call weather API here
    return f"Weather in {city}: 32 degrees {units}"

# LangChain reads the function signature automatically:
print(get_weather.name)          # "get_weather"
print(get_weather.description)   # the docstring above
print(get_weather.args_schema)   # JSON schema of the parameters
```

Note: The docstring IS the tool description the LLM sees. Write clear, specific descriptions — the LLM uses them to decide WHEN to use each tool.

3. Tool Closures — Passing State to Tools

A common problem: tools need access to shared state (a database client, an API key, an arq pool). But tool functions can only receive the arguments the LLM passes. The solution: define tools inside a factory function using **closures**.

```
from langchain_core.tools import tool
from supabase import Client

def make_tools(supabase: Client, arq_pool) -> list:
    """Factory function – builds tools that close over supabase and arq_pool."""

    @tool
    async def get_ideas(status: str = "pending_approval") -> str:
        """Browse content ideas by status."""
        # supabase is captured from the outer scope (closure)
        resp = supabase.table("ideas").eq("approval_status", status).execute()
        return str(resp.data)

    @tool
```

```

    async def trigger_research(topic: Optional[str] = None) -> str:
        """Trigger the research agent."""
        # arq_pool is captured from the outer scope
        job = await arq_pool.enqueue_job("research_agent_task", topic=topic)
        return f"Enqueued: {job.job_id}"

    return [get_ideas, trigger_research]

# Usage:
tools = make_tools(supabase_client, arq_pool_instance)

```

4. Binding Tools to a Claude Model (LangChain)

```

from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage

llm = ChatAnthropic(model="claude-sonnet-4-6", api_key="...")

# Bind tools to the model
tools = make_tools(supabase, arq_pool)
llm_with_tools = llm.bind_tools(tools)

# Now the LLM can decide to call tools
response = await llm_with_tools.ainvoke([
    HumanMessage(content="Show me the 5 top-scoring pending ideas for LinkedIn")
])

# If the LLM called a tool:
if response.tool_calls:
    for call in response.tool_calls:
        print(f"Tool: {call['name']}, Args: {call['args']}")

```

5. LangGraph — State Machine for Agents

LangGraph structures an agent as a directed graph. Each **node** is a function that processes the current state. **Edges** connect nodes — including **conditional edges** that route to different nodes based on the state.

```

from langgraph.graph import StateGraph, END
from langchain_core.messages import HumanMessage, AIMessage
from typing import TypedDict, Annotated
import operator

# Define the state – what persists across nodes
class AgentState(TypedDict):
    messages: Annotated[list, operator.add] # messages accumulate

# Build the graph
graph = StateGraph(AgentState)

# ■■ Node 1: the LLM decides what to do ■■
async def call_llm(state: AgentState) -> dict:
    response = await llm_with_tools.ainvoke(state["messages"])
    return {"messages": [response]}

```

```

# ■■ Node 2: execute the tool the LLM chose ■■
from langgraph.prebuilt import ToolNode
tool_node = ToolNode(tools)

# ■■ Conditional routing ■■
def should_use_tool(state: AgentState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "call_tool"    # LLM wants to use a tool
    return END                # LLM gave a final answer

# Wire up the graph
graph.add_node("llm", call_llm)
graph.add_node("tool", tool_node)
graph.set_entry_point("llm")
graph.add_conditional_edges("llm", should_use_tool, {"call_tool": "tool", END: END})
graph.add_edge("tool", "llm")    # after tool call, go back to LLM

agent = graph.compile()

# Run the agent
result = await agent.ainvoke({
    "messages": [HumanMessage(content="Show pending ideas and approve the top one")]
})

```

6. Memory and Conversation History

```

# In-memory checkpointer (lost on restart)
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver()
agent = graph.compile(checkpointer=checkpointer)

# Run with a thread_id – state is saved per thread
config = {"configurable": {"thread_id": "user_session_1"}}

await agent.ainvoke(
    {"messages": [HumanMessage("Show me pending ideas")]},
    config=config,
)
# Later in the same thread – agent remembers the conversation
await agent.ainvoke(
    {"messages": [HumanMessage("Approve the third one")]},
    config=config,
)

```

7. Human-in-the-Loop

LangGraph supports pausing the agent at a specific node to wait for human input. This is the pattern ContentAutomation2 uses for Gate 1 (idea approval) and Gate 2 (draft approval) — the agent presents results, a human reviews, then the agent continues.

```

# Add an interrupt before executing high-stakes tools
graph.compile(
    checkpointer=checkpointer,

```

```

        interrupt_before=["publish_to_twitter"],    # pause before this node
    )

# The agent runs until the interrupt, then you resume after human approval:
await agent.astream(messages, config)
# ... user reviews ...
await agent.astream(None, config)    # None = resume from where it stopped

```

8. How ContentAutomation2 Uses LangGraph

The orchestrator agent (`backend/app/agents/orchestrator/agent.py`) is a LangGraph graph with ~30 tools. The tools are built using the closure factory in `backend/app/agents/orchestrator/tools.py`. Each tool closes over the supabase client and `arq` pool. The frontend talks to the agent via a streaming WebSocket.

9. Practice Exercises

- Exercise 1: Create a `@tool` called `get_stock_price(ticker: str)` that returns a random price.
- Exercise 2: Create a second `@tool` called `buy_stock(ticker: str, shares: int)` that logs the purchase.
- Exercise 3: Use `make_tools()` factory pattern to pass a shared 'portfolio' dict to both tools.
- Exercise 4: Bind both tools to `ChatAnthropic`. Ask: 'What is the price of AAPL? Buy 10 shares if it's below 200.'
- Exercise 5: Build a LangGraph with 2 nodes: `call_llm` and `call_tool`. Add conditional routing.
- Exercise 6: Add a `MemorySaver` and test a 3-turn conversation where the agent remembers earlier turns.